

tenbin-contracts

Apex Security Review

May 5, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
4	Scan #1	4
4.1	High Risk	4
4.1.1	TENB1-7 - If staking supply reaches zero during vesting, the next depositor can steal the entire unvested reward bucket	4
4.1.2	TENB1-8 - Revenue withdrawals pay out fresh principal against unrealized vault gains	6
4.2	Medium Risk	8
4.2.1	TENB1-4 - Controller prices USDC units as if they were USD, so a below-peg USDC lets users mint unbacked tGLD	8
4.2.2	TENB1-10 - Live CollateralManager keeps stale pendingRevenue across collateral removal and re-add	10
4.2.3	TENB1-9 - Deployed v1.2.0 Controller shares one blockNumber across mint and redeem throttles, letting the first action of a block freeze the opposite side	11
4.2.4	TENB1-6 - Live stGLD cooldown state is storage-incompatible with controller-compatible staking upgrades, so a hotfix can orphan siloed assets	13
4.2.5	TENB1-5 - cancelCooldown can consume a user's cooled assets for zero shares in low-supply high-asset states	15
4.3	Low Risk	17
4.3.1	TENB1-2 - CI silently falls back to the default Foundry profile and never validates production bytecode	17
4.3.2	TENB1-11 - Live stGLD lets any approved spender indefinitely reset a user's entire cooldown with dust-sized withdrawals	18
4.3.3	TENB1-1 - Emergency collateral removal leaves mints live while redemptions are bricked	21
4.4	Informational Risk	22
4.4.1	TENB1-3 - README and docs point reviewers to obsolete mainnet Controller and RevenueModule contracts	22

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

This report summarizes the client-visible findings for tenbin-contracts for tenbin-contracts. Effective severities, statuses, notes, and comments are sourced from persisted client feedback at export time.

From May 1, 2026 to May 1, 2026 Cantina conducted a security review of [tenbin-contracts](#) on commit hash [99355b0540d89613a59f7ee15387ee396d84c0eb](#). The team identified a total of **11** issues:

Issues Found			
Severity	Count	Fixed	Acknowledged
Extreme Risk	0	0	0
Critical Risk	0	0	0
High Risk	2	0	0
Medium Risk	5	0	0
Low Risk	3	0	0
Informational	1	0	0
Total	11	0	0

Findings without a Fixed or Acknowledged client label remain Open and are listed in their respective finding sections.

Cantina reviewed [tenbin-contracts](#) holistically on commit hash [99355b0540d89613a59f7ee15387ee396d84c0eb](#). At the time of this export, the included findings remain open and no new vulnerabilities were identified.

2.1 Scope

The review covered the following scan snapshots:

- Scan #1
 - <https://github.com/tenbinlabs/tenbin-contracts> - branch master - commit 99355b0540d8

tenbin-contracts for tenbin-contracts was reviewed by Cantina across 1 scan.

This export includes 11 client-visible findings, grouped by effective severity and using persisted client statuses for report presentation.

3 Findings

4 Scan #1

Created: May 1, 2026

Repositories

- <https://github.com/tenbinlabs/tenbin-contracts> - branch master - commit 99355b0540d8

4.1 High Risk

4.1.1 TENB1-7 - If staking supply reaches zero during vesting, the next depositor can steal the entire unvested reward bucket



Status: Open

Original severity: High

Executive Summary StakedAsset can enter a state where `totalSupply() == 0` while the contract still holds a full bucket of unvested rewards. This happens because all exit paths only let users remove `totalAssets()`, and `totalAssets()` explicitly excludes `_pendingRewards()`. Once the last share is burned, the reward bucket remains stranded inside the vault with no outstanding supply attached to it.

At that point, the next depositor gets the best possible trade in ERC4626: the vault is economically full, but the accounting denominator is zero. OpenZeppelin's empty-vault bootstrap then mints shares 1:1 against `totalAssets() + 1`, so a dust deposit can capture essentially the entire stranded reward bucket after vesting completes. The same failure also occurs if the rewarder sends rewards while supply is already zero. This lets an arbitrary new entrant convert previously queued rewards into personal profit with trivial capital.

Details Reward vesting is tracked separately from shares:

```
function reward(uint256 assets) external onlyRole(REWARDER_ROLE) {
    if (vesting.period > 0) {
        uint256 pending = _pendingRewards();
        vesting.assets = pending + assets;
        vesting.end = uint128(block.timestamp) + vesting.period;
    }
    IERC20(asset()).safeTransferFrom(msg.sender, address(this), assets);
}

function totalAssets() public view override returns (uint256) {
    return IERC20(asset()).balanceOf(address(this)) - _pendingRewards();
}
```

All exit paths consume only `totalAssets()` and can therefore burn the final share while leaving the pending bucket behind:

```
function cooldownAssets(uint256 assets) external nonRestricted(msg.sender) returns (uint256 shares,
    uint256 id) {
    shares = previewWithdraw(assets);
    _withdraw(_msgSender(), address(silo), msg.sender, assets, shares);
}
```

After the last share exits, OpenZeppelin's mint math falls back to the empty-vault bootstrap:

```
function _convertToShares(uint256 assets, Math.Rounding rounding) internal view virtual returns (
    uint256) {
    return assets.mulDiv(totalSupply() + 10 ** _decimalsOffset(), totalAssets() + 1, rounding);
}
```

Concrete exploit:

- A sole or final staker holds 1,000 assets and the rewarder queues 1,000 more under vesting.
- `totalAssets()` is still 1,000, so the last staker can `redeem`, `withdraw`, or `cooldownAssets(1,000)` and burn all shares while removing only the vested principal.
- The vault now has `balance = 1,000`, `pendingRewards = 1,000`, `totalAssets = 0`, and `totalSupply = 0`.
- An attacker deposits 1 wei just before vesting ends and receives 1 share because the vault looks empty to ERC4626 pricing.
- When vesting completes, that 1 share redeems almost the full 1,000 stranded reward bucket.

This is a pure accounting break: the reward bucket is real, already transferred to the vault, and becomes claimable by whoever notices the empty-supply window first.

Impact Cascade

- Previously queued rewards become detached from the stakers they were meant to compensate.
- Any arbitrary depositor can steal the detached bucket with negligible capital once supply hits zero.
- The same issue appears if `reward()` is called while supply is already zero, so keeper automation can seed the trap accidentally.
- The stolen `tGLD` can then be redeemed externally for protocol value.

Assumptions and Uncertainties

- Supply can reach zero during vesting through normal user behavior; this is permissionless because `redeem`, `withdraw` (when `cooldown` is disabled), and both `cooldown` entrypoints all burn shares based on `totalAssets()`.
- There is at least one subsequent depositor after the pool empties.
- Mainnet currently uses a short vesting period, so the attacker only needs to monitor a relatively small time window.

How will the bug recipient respond? "If the pool is empty, there is no rightful recipient for those rewards, so letting the next staker claim them is acceptable."

That is still a broken accounting outcome. The protocol already minted/transferred real `tGLD` into the vault and marked it as a reward bucket. If no staker is entitled to it anymore, the correct behavior is to reject the reward, refund it, or keep it out of the ERC4626 asset pool entirely. Handing the entire bucket to the first dust depositor is arbitrary windfall creation and an exploitable leak.

Why did tests miss this issue? Existing tests assert `totalAssets() == balance - pendingRewards()` and cover ordinary `cooldown`/reward flows, but they never test the critical state combination `totalSupply() == 0 && pendingRewards() > 0`. Without that state, the empty-vault bootstrap path never gets exercised.

Recommendation Add a hard invariant that queued rewards cannot coexist with zero supply.

Concretely:

- Revert `reward()` when `totalSupply() == 0`.
- On every full exit, either distribute or explicitly refund any remaining pending rewards instead of leaving them inside the vault.
- As an additional guardrail, when `totalSupply() == 0`, price the first mint against the full asset balance rather than `balance - pendingRewards`.

References

- [tenbin-contracts/src/StakedAsset.sol#L183](#)
- [tenbin-contracts/src/StakedAsset.sol#L244](#)
- [tenbin-contracts/src/StakedAsset.sol#L383](#)
- [tenbin-contracts/lib/openzeppelin-contracts-upgradeable/contracts/token/ERC20/extensions/ERC4626Upgradeable.sol#L247](#)
- [tenbin-contracts/src/AssetSilo.sol#L44](#)

4.1.2 TENB1-8 - Revenue withdrawals pay out fresh principal against unrealized vault gains



Status: Open

Original severity: High

Executive Summary CollateralManager treats any increase in the manager's ERC4626 share value as immediately withdrawable revenue, but `withdrawRevenue()` never realizes that gain from the vault. Instead, it transfers whatever idle collateral currently happens to sit in the manager wallet. In practice, that means future user mints, curator withdrawals for redemption liquidity, or custodian top-ups can be paid out as "revenue" even though the actual gain is still trapped inside the vault.

This is not just a liquidity mismatch. Once `withdrawRevenue()` succeeds, it advances `lastTotalAssets` to the current vault valuation and reduces `pendingRevenue`, so any later vault loss is absorbed by backing collateral rather than clawing back the payout. The protocol has therefore converted a paper gain into an irreversible principal withdrawal. A later loss, bad debt event, adapter penalty, or any other negative vault repricing leaves outstanding `tGLD` undercollateralized by the amount previously extracted.

Details `getRevenue()` and `_getRevenue()` define revenue purely as the delta between the manager's current vault share value and `lastTotalAssets`:

```
function _getRevenue(address collateral, IERC4626 vault) internal view returns (uint256 revenue) {
    uint256 previousRevenue = pendingRevenue[collateral];
    uint256 totalAssets = _totalAssets(vault);
    uint256 lastTotal = lastTotalAssets[collateral];
    if (totalAssets > lastTotal) {
        uint256 gain = totalAssets - lastTotal;
        revenue = previousRevenue + gain;
    } else if (totalAssets < lastTotal) {
        uint256 loss = lastTotal - totalAssets;
        revenue = loss >= previousRevenue ? 0 : previousRevenue - loss;
    } else {
        revenue = previousRevenue;
    }
}
```

But `withdrawRevenue()` does not redeem any vault shares. It simply transfers idle tokens out of the manager:

```
function withdrawRevenue(address collateral, uint256 amount) external nonReentrant notPaused
onlyRevenueModule {
    IERC4626 vault = vaults[collateral];
    uint256 revenue = _getRevenue(collateral, vault);
    if (amount > revenue) revert ExceedsPendingRevenue();
    IERC20(collateral).safeTransfer(msg.sender, amount);
    pendingRevenue[collateral] = revenue - amount;
    lastTotalAssets[collateral] = _totalAssets(vault);
}
```

That disconnect creates the following exploitable accounting failure:

- Vault principal is 100, and its share value increases to 120, so `_getRevenue()` reports 20.
- A new user mint sends 20 fresh USDC into the manager wallet as backing collateral.
- The revenue keeper calls `collect(USDC, 20)`, which routes to `withdrawRevenue()` and transfers that fresh 20 USDC to RevenueModule.
- `lastTotalAssets` is then updated to 120, so the system considers the unrealized gain "consumed".
- If the vault later reprices back to 100 (loss, bad debt realization, adapter penalty, etc.), `_getRevenue()` falls to zero - but the 20 USDC already removed from the manager is not restored.

End state: the protocol now has 100 of remaining assets backing 120 of principal claims. The withdrawn amount came from a later user's principal, not from realized vault cashflow.

This is a live-path issue because the README explicitly expects revenue collection to happen independently from vault withdrawals, and the existing tests model that exact behavior. The implementation therefore relies on unrealized vault marks being risk-free and permanent, which is not guaranteed for ERC4626 integrations that explicitly support loss realization.

Impact Cascade

- Honest revenue collection can withdraw fresh user principal instead of realized yield.
- Later vault losses become a permanent shortfall instead of merely reducing unrealized revenue.
- The more often keepers “collect” while gains stay invested, the more incoming principal can be socialized into protocol expenses.
- A sufficiently large loss event can leave outstanding tGLD materially undercollateralized.

Assumptions and Uncertainties

- Can the reported vault gain later disappear? Yes. The Morpho V2 integration explicitly supports loss realization, and the accounting code already tries to handle negative deltas.
- Does `withdrawRevenue()` actually realize vault gains before paying them out? No. It only transfers the manager’s idle ERC20 balance.
- Does this require admin mistakes or a malicious vault choice? No. It only requires a production-feasible negative vault repricing after a keeper has collected against an unrealized gain.
- Is there any state that “undoes” the payout when the loss happens later? No. `lastTotalAssets` is advanced during collection, so the later loss only reduces future revenue.
- Would a triager argue this is “just timing” because total assets were sufficient at collection time? That defense fails because the withdrawn cash came from unrelated principal, while the claimed gain remained exposed to future loss.

How will the bug recipient respond? “Revenue is only collected when the vault has already earned the gain, so paying it from idle balance is economically equivalent.”

That argument only holds if the gain is immediately risk-free and cannot reverse. The code itself disproves that assumption: `_getRevenue()` has explicit loss-handling branches, meaning the integration acknowledges that vault marks can go down after collection. Once `withdrawRevenue()` has paid out from fresh principal, later losses are no longer netted against the withdrawn amount.

Why did tests miss this issue? The unit and invariant suites only test monotonic-gain scenarios: they mint interest into the vault, withdraw some idle liquidity, and then collect revenue. They never model the critical sequence of `collect revenue` -> `later vault loss`, so they miss the fact that `withdrawRevenue()` pays from unrelated principal and finalizes the mark before the loss occurs.

Recommendation The safest fix is to realize gains before paying them out. `withdrawRevenue()` should redeem or withdraw the requested amount from the vault (or from an explicitly tracked realized-revenue buffer) instead of transferring arbitrary idle collateral.

A robust pattern is:

```
uint256 revenue = _getRevenue(collateral, vault);
if (amount > revenue) revert ExceedsPendingRevenue();
vault.withdraw(amount, address(this), address(this));
pendingRevenue[collateral] = revenue - amount;
lastTotalAssets[collateral] = _totalAssets(vault);
IERC20(collateral).safeTransfer(msg.sender, amount);
```

If the protocol intentionally wants to keep gains invested, then revenue withdrawals must be blocked unless an explicit realized-revenue balance is available, and that balance must be debited on any later vault loss before principal is touched.

References

- `src/CollateralManager.sol#L302-L305`
- `src/CollateralManager.sol#L350-L357`
- `src/CollateralManager.sol#L439-L452`
- `README.md#L307-L319`
- `test/unit/CollateralManager.t.sol#L517-L547`
- `test/invariant/handlers/CollateralManagerHandler.sol#L181-L205`
- `src/CollateralManager.sol#L431-L433`

4.2 Medium Risk

4.2.1 TENB1-4 - Controller prices USDC units as if they were USD, so a below-peg USDC lets users mint unbacked tGLD

✓ Medium

Status: Open

Original severity: Medium

Executive Summary `Controller.verifyOrder()` treats the supported collateral amount as a USD amount after only decimal normalization. It never prices the collateral token itself. On mainnet the live controller accepts USDC (`isCollateral(USDC) == true`) and uses a 5% XAU/USD tolerance, so the backstop currently assumes 1 USDC == 1 USD at all times. If USDC trades below par, an attacker can buy discounted USDC, submit a mint order whose **nominal** USDC amount is within the 5% oracle band, and still mint full-value tGLD. The Chainlink gold price can be perfectly honest and the order will still pass, because the code compares `gold_usd_price` against `usdc_units_per_tGLD` rather than against the **real USD value** of the collateral. This produces directly undercollateralized tGLD on a live path, which can then be sold or later redeemed against healthy collateral.

Details The entire price check is performed by normalizing collateral token decimals to 18 and dividing by `order.asset_amount`:

```
uint256 oraclePrice = IOracleAdapter(oracle.adapter).getPrice();
uint256 decimals = IERC20Metadata(order.collateral_token).decimals();
uint256 collateralAmount;
if (decimals == 18) collateralAmount = order.collateral_amount;
else collateralAmount = order.collateral_amount * 10 ** (18 - decimals);

uint256 price = Math.mulDiv(collateralAmount, 1e18, order.asset_amount);
uint256 difference = price >= oraclePrice ? price - oraclePrice : oraclePrice - price;
uint256 delta = Math.mulDiv(difference, 1e18, oraclePrice);
if (delta > oracleData.tolerance) revert ExceedsOracleDeltaTolerance();
```

The oracle adapter returns only the gold price in USD:

```
(, int256 answer,, uint256 updatedAt,) = oracle.latestRoundData();
if (block.timestamp - updatedAt > PRICE_STALENESS_THRESHOLD) revert OraclePriceStale();
if (answer <= 0) revert InvalidOraclePrice();
return answer.toUint256() * DECIMALS_OFFSET;
```

No collateral/USD feed is consulted anywhere. The only collateral-specific logic is the `decimals()` call and the onboarding range check in `setIsCollateral()`.

On mainnet today, the live controller at `0xF24ceF8227A3126e49Ff272cDC06FC16A97c8967` reports (`adapter=0xA944664E98FF8CD1743149A62Da3F3F23297E7C1`, `tolerance=5e16`) and `isCollateral(USDC) == true`. That means the live safety band currently allows nominal order prices from 95% to 105% of spot.

Concrete exploit:

- Assume Chainlink reports gold at \$2,300 and USDC trades at \$0.90.
- The attacker buys 2,185 USDC for \$1,966.50.
- The attacker submits a mint for 1 tGLD with `collateral_amount = 2,185e6` and `asset_amount = 1e18`.
- `verifyOrder()` computes `price = 2,185e18`, so `delta = (2300 - 2185) / 2300 = 5%`, which passes the live tolerance exactly.
- `mint()` then transfers the full 2,185 USDC nominal amount and mints 1 tGLD:

```
IERC20(order.collateral_token).safeTransferFrom(order.payer, custodian, custodianAmount);
IERC20(order.collateral_token).safeTransferFrom(order.payer, manager, order.collateral_amount - custodianAmount);
AssetToken(asset).mint(order.recipient, order.asset_amount);
```

Economically, the protocol has accepted only \$1,966.50 of real value for an asset worth \$2,300.

Root cause: the controller assumes supported collateral is worth one dollar per token unit, but the oracle it compares against is a USD price oracle for gold, not a price oracle for the collateral.

Impact Cascade

- The attacker mints tGLD with materially less real USD value than the order guard is supposed to allow.
- The resulting tGLD supply is undercollateralized at creation time.
- The attacker can immediately sell the excess tGLD or hold it until collateral conditions normalize.
- Any later redemption converts this bad debt into a direct drain on healthy backing held by the manager/custodian.
- During a stablecoin stress event, repeated mints can scale the insolvency quickly because the live tolerance still accepts nominal prices within 5%.

Assumptions and Uncertainties

- The live system continues approving ordinary mint orders while the collateral token trades materially below par.
- The attacker can source discounted supported collateral on the open market, which is realistic for USDC-style depeg scenarios.
- The issue does not require oracle manipulation; it relies on honest XAU/USD data and honest collateral token balances.
- If operators manually pause minting instantly on every depeg, the exploit window shrinks, but the on-chain backstop still fails at its stated job.
- The impact scales with the size and duration of the collateral depeg and with how many approved orders are processed before operations halt.

How will the bug recipient respond? "We only support reputable dollar stablecoins, and operations would pause if USDC depegged."

That response does not address the code flaw. The contract advertises the oracle as a protection against off-spot minting, but the implementation never prices the collateral in USD. A stablecoin depeg is exactly the type of market dislocation where the backstop should matter most. Relying on manual intervention leaves a live critical path unprotected.

Why did tests miss this issue? The oracle tests only vary token decimals and the gold oracle answer. They always assume the collateral token itself is worth exactly one dollar, so they never model the economically relevant case where `collateral_token` units diverge from USD value.

Recommendation Primary fix: validate orders against the **USD value of collateral**, not just the collateral token amount. This requires either:

- a collateral/USD oracle per supported collateral; or
- hard support only for collateral whose price is itself oracle-checked and bounded on-chain.

A minimal design sketch is:

```
uint256 collateralUsd = Math.mulDiv(order.collateral_amount, collateralOraclePrice, 10 **
    collateralDecimals);
uint256 orderPrice = Math.mulDiv(collateralUsd, 1e18, order.asset_amount);
```

Also add an emergency guard that pauses mints if the collateral oracle deviates from \$1 beyond a configured bound.

References

- [src/Controller.sol#L243-L251](#)
- [src/Controller.sol#L381-L394](#)
- [src/Controller.sol#L490-L540](#)
- [src/oracle/GoldOracleAdapter.sol#L16-L39](#)

4.2.2 TENB1-10 - Live CollateralManager keeps stale pendingRevenue across collateral removal and re-add

✓ Medium

Status: Open

Original severity: Medium

Executive Summary The mainnet CollateralManager implementation at 0x1b1ed1db1038b4d2d9b2b3f35d3b0dd82e8438 omits the `pendingRevenue[collateral] = 0` reset inside `removeCollateral()`, even though the function comment says removing support should “give up any pending revenue” for that collateral. As a result, a previously realized revenue balance survives an emergency collateral removal and then reattaches to the next vault generation if the same collateral is later re-added.

Once that stale state exists, `withdrawRevenue()` will treat the carried-over amount as fresh withdrawable revenue for the re-added collateral and can pay it out of newly introduced backing principal. A `REVENUE_KEEPER_ROLE` holder can therefore extract fresh collateral after an otherwise legitimate admin migration / emergency-removal flow, without needing `REBALANCER_ROLE` and without any new yield having been earned in the replacement vault.

Details The deployed implementation’s `removeCollateral()` clears approvals and `lastTotalAssets`, but does **not** clear `pendingRevenue`:

```
function removeCollateral(address collateral) external onlyRole(DEFAULT_ADMIN_ROLE) {
    IERC4626 vault = vaults[collateral];
    if (address(vault) == address(0)) revert CollateralNotSupported();
    IERC20(collateral).forceApprove(controller, 0);
    IERC20(collateral).forceApprove(address(vault), 0);
    lastTotalAssets[collateral] = 0;
    collaterals.remove(collateral);
    delete vaults[collateral];
    emit CollateralRemoved(collateral, address(vault));
}
```

But the checked-in repo remediation explicitly adds the missing reset and its unit test asserts that removal must zero out pending revenue:

```
IERC20(collateral).forceApprove(controller, 0);
IERC20(collateral).forceApprove(address(vault), 0);
pendingRevenue[collateral] = 0;
lastTotalAssets[collateral] = 0;
```

```
manager.removeCollateral(address(collateral));
assertEq(manager.pendingRevenue(address(collateral)), 0);
```

The exploit path is:

- The manager has a nonzero `pendingRevenue[collateral]` for a supported collateral after prior realized yield.
- Admin performs the documented emergency / migration flow and calls `removeCollateral(collateral)` on the live deployment.
- Because the live contract omits the reset, the old `pendingRevenue[collateral]` survives even though the old vault is no longer supported.
- Admin later re-adds the same collateral with a replacement vault via `addCollateral(collateral, new Vault)`. This initializes only `lastTotalAssets`; it does not clear the stale `pendingRevenue`.
- After ordinary operations reintroduce fresh idle collateral for that token (for example future mint liquidity or curator-managed vault withdrawals), `withdrawRevenue(collateral, amount)` succeeds against `_getRevenue()`, which still includes the stale carried-over `pendingRevenue`.
- The revenue module receives fresh collateral from the new vault generation even though that amount belonged to backing principal, not new revenue.

The root cause is that revenue state is keyed only by collateral address, but `removeCollateral()` does not destroy that state when the collateral/vault lifecycle is reset. The next vault generation inherits an accounting claim from the old one.

Impact Cascade

- A routine admin collateral migration or emergency vault removal can leave a latent stale-revenue claim on the removed collateral address.
- After the collateral is re-added, `REVENUE_KEEPER_ROLE` can withdraw that stale amount from fresh backing principal.
- The replacement vault / manager balance loses redeemable collateral even though no equivalent new yield was earned there.
- The repo's own remediation and tests show this was already recognized as unsafe, but the mainnet deployment still lacks the fix.

Assumptions and Uncertainties

- The same collateral address is re-added after removal. This is realistic for a vault migration or emergency replacement on a live USDC-backed system.
- A nonzero `pendingRevenue[collateral]` exists before removal. This is a normal reachable state after realized yield has been recorded.
- Fresh idle collateral becomes available before collection. This is consistent with ordinary mint / withdrawal liquidity management and with the existing revenue-withdraw design.

How will the bug recipient respond? "This needs an admin migration before the stale state exists, so it is just an operational concern."

That response is weaker than the bug. The admin action is the documented, intended `removeCollateral()` lifecycle, not a mistake, and the live repo already contains a targeted remediation plus a dedicated test asserting that pending revenue must be zeroed on removal. The vulnerable state is therefore created by an otherwise legitimate maintenance path, after which a revenue keeper can extract principal that should never have been collectible.

Why did tests miss this issue? The current repo tests do not miss it; they explicitly assert the fixed behavior. The issue exists because mainnet is still running an older implementation that predates the remediation.

Recommendation Upgrade the live `CollateralManager` implementation so that `removeCollateral()` clears all revenue state for the removed collateral:

```
pendingRevenue[collateral] = 0;
lastTotalAssets[collateral] = 0;
```

As a defense in depth, consider versioning revenue state by vault address or forbidding `addCollateral()` from reusing a collateral address while stale accounting remains.

References

- [tmp/d8048c8e-9330-49ca-9d79-32f88d55758c/live_CollateralManager.sol#L158-L167](#)
- [tmp/d8048c8e-9330-49ca-9d79-32f88d55758c/live_CollateralManager.sol#L335](#)
- [tmp/d8048c8e-9330-49ca-9d79-32f88d55758c/live_CollateralManager.sol#L435](#)
- [tenbin-contracts/src/CollateralManager.sol#L172-L175](#)
- [tenbin-contracts/test/unit/CollateralManager.t.sol#L352-L353](#)

4.2.3 TENB1-9 - Deployed v1.2.0 Controller shares one `blockNumber` across mint and redeem throttles, letting the first action of a block freeze the opposite side

✓ **Medium**

Status: Open

Original severity: Medium

Shared '`blockNumber`' cross-couples mint and redeem limits

Executive Summary The live mainnet Tenbin deployment is still on protocol version 1.2.0, as shown in `deployments.json`, and the deployed controller at `0xF24ceF8227A3126e49Ff272cDC06FC16A97c8967` reports the same version on-chain. In that version, the per-block mint and redeem throttles use separate amount accumulators (`blockMints` and `blockRedeems`) but a single shared `blockNumber`. As a result, whichever side executes first in a new block refreshes the shared block marker for both throttles while leaving the opposite-side accumulator stale. A user who saturates one limit in block `N` can therefore submit a tiny transaction

of the opposite type at the start of block $N+1$ and make every legitimate transaction of the original type in that block compare against the previous block's saturated counter.

This turns the controller's rate limiter into a cross-coupled freeze primitive. A trader with enough inventory to fill one side's public limit can keep mints or redemptions reverting block after block, causing live redemption breakage and issuance outages without needing any privileged role.

Details The deployed v1.2.0 controller stores:

```
uint256 public blockMintLimit;
uint256 public blockRedeemLimit;
uint256 blockMints;
uint256 blockRedeems;
uint256 blockNumber;
```

In mint, the limiter resets only blockMints but also overwrites the shared blockNumber:

```
uint256 limit = blockMintLimit;
if (limit < type(uint256).max) {
    if (block.number > blockNumber) {
        if (order.asset_amount > limit) revert ExceedsBlockMintLimit();
        blockMints = order.asset_amount;
        blockNumber = block.number;
    } else {
        uint256 newMints = blockMints + order.asset_amount;
        if (newMints > limit) revert ExceedsBlockMintLimit();
        blockMints = newMints;
    }
}
```

redeem mirrors that logic, but again keys its "new block" test off the same global blockNumber:

```
uint256 limit = blockRedeemLimit;
if (limit < type(uint256).max) {
    if (block.number > blockNumber) {
        if (order.asset_amount > limit) revert ExceedsBlockRedeemLimit();
        blockRedeems = order.asset_amount;
        blockNumber = block.number;
    } else {
        uint256 newRedeems = blockRedeems + order.asset_amount;
        if (newRedeems > limit) revert ExceedsBlockRedeemLimit();
        blockRedeems = newRedeems;
    }
}
```

The bug appears when the two flows interleave across adjacent blocks:

- In block N , the attacker fills the redeem limit, so `blockRedeems == blockRedeemLimit`.
- In block $N+1$, the attacker submits a tiny mint before any legitimate redeem.
- `mint()` sees a new block, resets `blockMints`, and updates the shared `blockNumber = N+1`, but it does **not** reset `blockRedeems`.
- Every later `redeem()` in the same block now enters the `else` branch because `block.number == blockNumber`, so it adds its amount to the stale `blockRedeems` value from block N and reverts immediately.

The same freeze exists in the opposite direction by saturating mints in one block and sending a dust redeem first in the next block.

Impact Cascade

- A public trader can carry a saturated redeem counter forward and make all redemptions in the next block revert.
- The same technique can freeze minting by carrying a saturated mint counter forward with a first redeem.
- The bug converts the rate limiter into a cross-flow griefing primitive, even though the two public limits are supposed to be independent.
- Repeating the pattern block-by-block creates sustained issuance or redemption outages on the live controller.

Assumptions and Uncertainties

- The deployed controller is the v1.2.0 implementation described by the checked-in deployment meta-data; I confirmed the live controller currently reports `version() = "1.2.0"`.
- The attacker can source enough inventory to fill one public 3000 tGLD-side block limit; this is capital-intensive but permissionless.
- The attacker only needs to win the first transaction of the following block with a dust-sized opposite-side order.

How will the bug recipient respond? "This only causes temporary throttling, not theft, and the attacker must already consume an entire block limit first."

That response understates the issue: the controller's documented safety rail is supposed to cap each side independently, not let one side suppress the other. Converting a public rate limit into a repeatable issuance/redemption freeze is a live liveness failure in the exact mint/redeem path the protocol relies on.

Why did tests miss this issue? The unit tests exercise mint and redeem limit enforcement independently, but they do not model the cross-block sequencing where one flow is first in a new block and the opposite flow then reuses the stale accumulator from the previous block.

Recommendation Use independent block markers for mint and redeem accounting.

```
uint256 mintBlockNumber;  
uint256 redeemBlockNumber;
```

Each limiter should compare and reset against its own block number, or the contract should use a struct per limiter (as the newer codebase already does).

References

- [deployments.json#L16-L38](#)
- [src/Controller.sol @ v1.2.0#L142-L154](#)
- [src/Controller.sol @ v1.2.0#L332-L345](#)
- [src/Controller.sol @ v1.2.0#L402-L415](#)
- [src/Controller.sol#L142-L146](#)
- [src/Controller.sol#L332-L343](#)
- [src/Controller.sol#L402-L413](#)

4.2.4 TENB1-6 - Live stGLD cooldown state is storage-incompatible with controller-compatible staking upgrades, so a hotfix can orphan siloed assets

✓ **Medium**

Status: Open

Original severity: Medium

Executive Summary The live stGLD proxy at `0xdE80e9EC32249d4c7dBA7997fD6D6C03fb27EBf4` is still running the old v1.0.0 cooldown layout, where cooldown state is stored as `mapping(address => Cooldown) cooldowns`. The controller-compatible staking versions in this repository (v1.2.0 and current HEAD) changed that layout to `mapping(address => mapping(uint256 => Cooldown)) cooldowns`, inserted `cooldownIds`, and shifted later state slots such as `cooldownPeriod`, `vesting`, and `isRestricted`. This makes an in-place UUPS upgrade of the live staking proxy storage-incompatible. Mainnet already has an active live cooldown: `cooldowns(0x2ffdb0d1381d6a0811095790951b98d51a2183ae)` returns 43404443433997586 assets and the silo holds exactly that amount. Under the new layout, the corresponding nested cooldown slot is zero. So a straightforward staking upgrade to restore controller compatibility would immediately orphan live siloed assets and corrupt other staking configuration/state. Any stGLD holder can also front-run a planned upgrade by entering cooldown first, forcing operators either to delay the hotfix or to strand funds.

Details The live staking proxy exposes the old single-cooldown ABI, not the newer id-based layout:

```
// v1.0.0 live layout  
AssetSilo public silo;  
mapping(address => Cooldown) public cooldowns;  
uint256 public cooldownPeriod;  
Vesting public vesting;  
mapping(address => bool) public isRestricted;
```


- A fresh-deployment migration avoids the issue; this finding is specifically about the unsafe in-place upgrade path exposed by the current storage layout change.

How will the bug recipient respond? "We can just wait five minutes for cooldowns to clear before upgrading."

That does not resolve the issue. There is no pause or gate on cooldown entry in the live staking contract, so any stGLD holder can front-run a known upgrade window by creating a new cooldown. More importantly, the layout mismatch also shifts non-cooldown state like `cooldownPeriod` and `vesting`, so the upgrade is unsafe even beyond the specific active cooldown records.

Why did tests miss this issue? The repository tests operate on a single source tree at a time and do not model upgrading a live v1.0.0 staking proxy into the later id-based cooldown implementations. The failure only appears when you combine live mainnet storage with the newer code's changed storage layout and interface assumptions.

Recommendation Do **not** perform an in-place UUPS upgrade of the current live staking proxy directly to the v1.2.0+ cooldown model.

Safer options are:

```
// preferred direction
// 1. deploy a fresh staking contract with the new layout
// 2. migrate balances/cooldowns explicitly, or require users to exit old staking first
// 3. point any new controller logic at the fresh staking deployment
```

If an in-place migration is unavoidable, introduce an explicit transition implementation that:

- reads legacy `cooldowns(address)` state,
- preserves or drains siloed assets before switching layouts,
- remaps `cooldownPeriod`, `vesting`, and `isRestricted` into the new slots,
- and only then upgrades to the new id-based implementation.

Also add deployment-time version/interface checks so the controller cannot be wired to an incompatible staking implementation again.

References

- `src/StakedAsset.sol#L82-L94` (live 'v1.0.0' layout, repo commit '34d0d98')
- `src/StakedAsset.sol#L93-L118` (controller-compatible later layout, repo commit '17b0193' / current 'HEAD')
- `README.md#L318` (staking documented as UUPS-upgradeable)

4.2.5 TENB1-5 - cancelCooldown can consume a user's cooled assets for zero shares in low-supply high-asset states

✓ **Medium**

Status: Open

Original severity: Medium

Executive Summary `StakedAsset.cancelCooldown()` deletes the caller's cooldown record and then blindly forwards the cooled assets into `AssetSilo.cancel()`, which in turn calls `ERC4626.deposit()` with no minimum-share check. OpenZeppelin's ERC4626 share conversion rounds down. In any low-supply / high-asset state - for example after a direct donation into staking or after rewards have accumulated while supply is very thin - `deposit(amount)` can legally mint 0 shares. When that happens during `cancelCooldown()`, the user's cooled `AssetToken` balance is transferred out of the silo and into the staking contract, the cooldown is already cleared, and the user receives nothing back. This is an irreversible direct-loss path on an end-user action that is supposed to be value-preserving.

Details The cooldown cancellation flow performs no slippage or minimum-share validation:

```
function cancelCooldown(uint256 id) external returns (uint256 shares) {
    Cooldown memory cooldown = cooldowns[msg.sender][id];
    uint256 assets = uint256(cooldown.assets);
    if (assets == 0) revert ZeroCooldownAssets();
    delete cooldowns[msg.sender][id];
    shares = silo.cancel(msg.sender, assets);
}
```


The silo simply routes those assets into a normal ERC4626 deposit:

```
function cancel(address account, uint256 amount) external returns (uint256 shares) {
    if (msg.sender != staking) revert OnlyStaking();
    return IERC4626(staking).deposit(amount, account);
}
```

OpenZeppelin's share minting rounds down:

```
function previewDeposit(uint256 assets) public view virtual returns (uint256) {
    return _convertToShares(assets, Math.Rounding.Floor);
}

function _convertToShares(uint256 assets, Math.Rounding rounding) internal view returns (uint256) {
    return assets.mulDiv(totalSupply() + 10 ** _decimalsOffset(), totalAssets() + 1, rounding);
}
```

StakedAsset.totalAssets() also excludes pending rewards:

```
function totalAssets() public view override returns (uint256) {
    return IERC20(asset()).balanceOf(address(this)) - _pendingRewards();
}
```

In a state where totalAssets() is high relative to totalSupply() - which can happen after donations, or after rewards accumulate while supply is thin - the floor rounding in _convertToShares() can return 0. cancelCooldown() treats that as success. The cooled assets leave the silo, the cooldown record is gone, and the user gets no shares back.

Impact Cascade

- A user can lose the entire cooled position on a supposedly reversible cancelCooldown() action.
- The loss is irreversible because the cooldown entry is deleted and the assets are now ordinary staking balance owned by the vault.
- Thin-liquidity or donated-balance states become dangerous for end users rather than just distorted pricing states.
- The bug compounds the existing low-supply ERC4626 edge cases by turning them into an immediate user-loss path.

Assumptions and Uncertainties

- The issue requires a low-supply / high-asset ratio, which can arise from direct donations or from protocol-generated reward states when the pool is thin.
- The deposit must round down to zero shares; that is permitted by the inherited ERC4626 implementation and there is no caller-specified minimum.
- This is not a revert-path concern: if the share mint rounds to zero, the transaction succeeds and the loss is finalized.

How will the bug recipient respond? "The user can just avoid canceling in those edge states, or wait and unstake instead."

That is not a sufficient defense. cancelCooldown() is documented as the value-preserving way to reverse a cooldown, but the implementation does not protect the caller against zero-share mints. A public user action should not silently destroy the user's claim because the vault ratio drifted into a thin-supply state.

Why did tests miss this issue? The cooldown tests cancel in normal states where the share/asset ratio is close to 1 and shares are guaranteed to mint. The suite never creates a high-asset / low-supply state and then exercises cancelCooldown() with a position small enough to round to zero shares.

Recommendation Add a minimum-share guard to the cancellation path and revert if the deposit would mint zero or unexpectedly few shares.

```
function cancelCooldown(uint256 id) external returns (uint256 shares) {
    ...
    shares = previewDeposit(assets);
    if (shares == 0) revert InvalidCooldownAmount();
    delete cooldowns[msg.sender][id];
    shares = silo.cancel(msg.sender, assets);
}
```

More generally, harden the low-supply ERC4626 state by overriding `_decimalsOffset()` or otherwise preventing thin-supply / high-asset states from reaching user-facing flows.

References

- `src/StakedAsset.sol#L201-L208`
- `src/StakedAsset.sol#L244-L253`
- `src/StakedAsset.sol#L383-L386`
- `src/AssetSilo.sol#L40-L47`
- `lib/openzeppelin-contracts/contracts/token/ERC20/extensions/ERC4626.sol#L151-L171`
- `lib/openzeppelin-contracts/contracts/token/ERC20/extensions/ERC4626.sol#L225-L248`
- `lib/openzeppelin-contracts-upgradeable/contracts/token/ERC20/extensions/ERC4626Upgradeable.sol#L247-L253`
- `src/StakedAsset.sol#L385-L387`

4.3 Low Risk

4.3.1 TENB1-2 - CI silently falls back to the default Foundry profile and never validates production bytecode

✓ **Low**

Status: Open

Original severity: Low

Executive Summary The repository documents and deploys production contracts with `FOUNDRY_PROFILE=production`, where `via_ir = true` and `optimizer_runs = 18888`. However, the GitHub Actions workflow sets `FOUNDRY_PROFILE=ci` even though `foundry.toml` defines no `ci` profile. Foundry therefore falls back to the default profile, which uses materially different compiler settings (`via_ir = false`, `optimizer_runs = 1000`). In practice, this means the project's canonical automated checks do not compile or test the same bytecode that is intended for production deployment.

This is not a theoretical style issue. I verified it locally with `forge config: FOUNDRY_PROFILE=ci` emits `Warning: Selected profile 'ci' does not exist; falling back to the default profile`. The resulting config matches the default profile rather than the production one. For a protocol whose live security depends on controller, collateral-manager, staking, and revenue-module bytecode, this invalidates CI as a source of assurance for production deployments.

Details The repo defines production-specific compiler settings here:

```
[profile.production]
src = "src"
skip = ["test/**/*"]
via_ir = true
ffi = true
optimizer = true
optimizer_runs = 18888
```

But CI explicitly requests a non-existent profile:

```
env:
  FOUNDRY_PROFILE: ci
```

and then uses that profile for both build and test:

```
- name: Run Forge build
  run: forge build

- name: Run Forge tests
  run: forge test -vvv
```

I verified the behavior with a read-only command:

```
FOUNDRY_PROFILE=ci forge config --json
Warning: Selected profile `ci` does not exist; falling back to the default profile.
via_ir = false
optimizer_runs = 1000
```

while `FOUNDRY_PROFILE=production forge config --json` reports `via_ir = true` and `optimizer_runs = 18888`.

The root cause is a broken assumption that CI is exercising a dedicated profile, when in reality it is silently testing the wrong compiler pipeline.

Impact Cascade

- CI greenlights artifacts compiled under different optimizer and IR settings than the documented production build.
- Reviewers can trust passing tests that say nothing about the deployed production bytecode.
- Any optimizer- or `via-ir`-sensitive behavior can ship to production without ever being exercised in the canonical workflow.
- The repo's public assurance story ("builds/tests passed") no longer applies to the bytecode operators are asked to deploy.

Assumptions and Uncertainties

- Production deployments follow the documented `FOUNDRY_PROFILE=production` flow in the README.
- Compiler pipeline differences (`via-ir`, optimizer runs, FFI/skip settings) are security-relevant for review integrity even without naming a specific miscompilation.
- This is a hygiene/integrity issue affecting confidence in production review, not an on-chain exploit path by itself.

How will the bug recipient respond? "The default profile is close enough, and the contracts still compile."

That misses the problem. The repository explicitly treats `production` as special, and production deployments are documented to use that profile. Once CI silently validates a different profile, the team no longer has automated assurance over the bytecode it intends others to trust and deploy.

Why did tests miss this issue? The workflow still returns green because Foundry treats the bad profile name as a warning, not an error. There is no CI assertion that the selected profile exists or that CI settings match the production compiler configuration.

Recommendation Either define `[profile.ci]` explicitly or change the workflow to use `FOUNDRY_PROFILE=production` for production-parity checks. Also add a guard step that fails if `forge config` reports a missing profile or if CI compiler settings diverge from the intended production build.

References

- [foundry.toml#L1](#)
- [foundry.toml#L28](#)
- [foundry.toml#L29-L35](#)
- [README.md#L61](#)
- [README.md#L63](#)
- [.github/workflows/contracts.yaml#L10-L11](#)
- [.github/workflows/contracts.yaml#L32-L36](#)

4.3.2 TENB1-11 - Live stGLD lets any approved spender indefinitely reset a user's entire cooldown with dust-sized withdrawals



Low

Status: Open

Original severity: Low

Executive Summary The live mainnet stGLD implementation at `0xdE80e9EC32249d4c7dBA7997fD6D6C03fb27EBf4` exposes `cooldownShares(address owner, uint256 shares)` and `cooldownAssets(address owner, uint256 assets)` to any caller, not just the owner whose shares are being cooled. Those functions use ERC4626's standard allowance path, so any spender with share allowance from `owner` can start cooldown on the owner's behalf. The critical flaw is that the live contract stores only one aggregated `cooldowns[owner]` entry and overwrites its `end` timestamp on every new cooldown. As a result, a malicious spender with even a tiny remaining allowance can wait until just before a victim's cooldown expires, trigger a dust-sized new cooldown from the victim's still-staked balance, and reset the unlock time for the victim's entire already-cooled amount. By repeating that transaction every cooldown window, the spender can keep the victim's queued withdrawal locked indefinitely while consuming negligible additional approved shares.

Details The live staking implementation uses a single cooldown slot per owner and lets arbitrary callers operate on another account's shares:

```

function cooldownShares(address owner, uint256 shares)
    external
    nonRestricted(msg.sender)
    nonRestricted(owner)
    returns (uint256 assets)
{
    if (shares == 0) revert InvalidCooldownAmount();
    if (shares > maxRedeem(owner)) revert CooldownExceededMaxRedeem();
    assets = previewRedeem(shares);
    cooldowns[owner].assets += assets.toUint160();
    cooldowns[owner].end = (block.timestamp + cooldownPeriod).toUint96();
    _withdraw(_msgSender(), address(silo), owner, assets, shares);
    emit CooldownStarted(owner, assets);
}

```

```

function cooldownAssets(address owner, uint256 assets)
    external
    nonRestricted(msg.sender)
    nonRestricted(owner)
    returns (uint256 shares)
{
    if (assets == 0) revert InvalidCooldownAmount();
    if (assets > maxWithdraw(owner)) revert CooldownExceededMaxWithdraw();
    shares = previewWithdraw(assets);
    cooldowns[owner].assets += assets.toUint160();
    cooldowns[owner].end = (block.timestamp + cooldownPeriod).toUint96();
    _withdraw(_msgSender(), address(silo), owner, assets, shares);
    emit CooldownStarted(owner, assets);
}

```

The `_withdraw` helper inherited from ERC4626 spends allowance whenever `caller != owner`:

```

function _withdraw(
    address caller,
    address receiver,
    address owner,
    uint256 assets,
    uint256 shares
) internal virtual {
    if (caller != owner) {
        _spendAllowance(owner, caller, shares);
    }
    _burn(owner, shares);
    SafeERC20.safeTransfer(IERC20(asset()), receiver, assets);
}

```

This creates the following live exploit path:

- A victim stakes `stGLD`, cools a large amount for withdrawal, but keeps even a tiny residual share balance staked.
- The victim has previously granted a spender allowance on `stGLD` (for example an unlimited approval to a dApp or automation contract).
- Just before the victim's cooldown expires, the malicious spender calls `cooldownShares(victim, 1)` or another dust-sized amount it is allowed to spend.
- The call only burns that dust from the victim's remaining liquid shares, but it updates `cooldowns[victim].end = block.timestamp + cooldownPeriod` for the victim's entire aggregated cooldown balance.
- The victim can no longer call `unstake()` for the previously queued amount until the new timestamp passes.
- The attacker repeats the same dust-sized call every cooldown window, keeping the whole queued withdrawal locked indefinitely.

The key authorization failure is that a small ERC20 allowance is supposed to let the spender move only the approved amount. Here it also gives the spender timing control over the owner's entire pre-existing cooldown queue, which can be arbitrarily larger than the newly spent dust.

Impact Cascade

- Any spender with standing stGLD allowance can block a victim's queued unstake indefinitely, as long as the victim retains some liquid shares.
- The attacker only needs dust-sized additional allowance consumption each cycle, while the affected locked amount can be the victim's full queued withdrawal.
- Victims lose the ability to exit on schedule through the live staking path, even though the spender was never approved to control the timing of previously cooled assets.
- The current mainnet cooldown is only 300 seconds, so an automated spender can maintain the lock with cheap repeated transactions.

Assumptions and Uncertainties

- The victim must have granted the attacker (or an attacker-controlled contract) stGLD allowance and must retain some unstaked stGLD shares after entering cooldown.
- If the victim cools their entire balance and leaves no remaining shares, the attacker cannot refresh the cooldown further because there are no shares left to spend.
- This is a live-path issue in the staking subsystem itself; it does not depend on admin compromise, oracle manipulation, or non-standard token behavior.

How will the bug recipient respond? "An approved spender is already allowed to withdraw the approved number of shares, so starting cooldown on behalf of the owner is within the allowance model."

That response misses the actual amplification. A spender approved for only a dust amount is not merely moving that dust: because the live contract aggregates all cooldown into one slot and resets the shared end timestamp, the spender can repeatedly relock an arbitrarily larger amount that had already been cooled earlier. Standard ERC20 allowance does not imply authority to control the withdrawal timing of unrelated, previously-withdrawn assets.

Why did tests miss this issue? The checked-in tests only exercise the newer owner-only, per-ID cooldown design in `tenbin-contracts/src/StakedAsset.sol`, where each cooldown entry has its own ID and callers can only cool their own shares. The live mainnet implementation is older and structurally different: it keeps a single `cooldowns[owner]` entry and accepts `owner` as a parameter. That design difference is exactly what enables the shared-timestamp reset.

Recommendation The minimal hotfix is to remove third-party cooldown initiation and require `msg.sender == owner` for cooldown entry, matching the current repo design. In parallel, stop using one aggregated cooldown slot per owner; keep separate cooldown entries so a new cooldown cannot reset the unlock time of previously queued assets.

A direct fix pattern already exists in the current repository:

```
function cooldownShares(uint256 shares) external nonRestricted(msg.sender) returns (uint256 assets,
    uint256 id) {
    id = cooldownIds[msg.sender]++;
    Cooldown storage cooldown = cooldowns[msg.sender][id];
    cooldown.end = cooldownEnd.toUint96();
    _withdraw(_msgSender(), address(silo), msg.sender, assets, shares);
}
```

References

- `tmp/live_StakedAsset.sol#L143-L171`
- `home/user/workspace/tenbin-contracts/lib/openzeppelin-contracts-upgradeable/contracts/token/ERC20/extensions/ERC4626Upgradeable.sol#L278-L299`
- `home/user/workspace/tenbin-contracts/src/StakedAsset.sol#L165-L197`
- `tenbin-contracts/src/StakedAsset.sol`

4.3.3 TENB1-1 - Emergency collateral removal leaves mints live while redemptions are bricked

✓ Low

Status: Open

Original severity: Low

Executive Summary is documented and implemented as the emergency response for a vault malfunction, but it only clears manager-side state. It revokes the controller's ERC20 allowance and deletes the manager's vault entry, yet it does not disable the same collateral inside. The result is a broken split-brain state: with still succeeds for that collateral because the controller only checks its own mapping before pulling funds directly from the payer into the custodian and manager, while is now bricked because manager approvals were zeroed and curated withdrawals revert as unsupported. On mainnet this matters because USDC is the sole live collateral. If operators invoke during a vault emergency, any user still holding an unexpired signed mint order can keep minting fresh into a market whose redemption leg has already failed, worsening the incident exactly when the emergency path is supposed to contain it.

Details is explicitly described as an emergency vault-malfunction function. But the implementation only updates the manager:

The controller's collateral registry is separate, and continues to accept the collateral until someone independently calls :

That desynchronization creates asymmetric order behavior.

Mints remain live. does not consult unless the order is curated. For ordinary uncurated mints it only verifies the controller-side collateral flag, then transfers funds directly from the payer to the configured custodian and manager and mints assets:

Redemptions, however, are broken immediately after. Curated redemptions fail because now sees the collateral as unsupported. Non-curated redemptions fail because has already revoked the manager's controller allowance, yet still tries to pull collateral from the manager:

This means the emergency action can produce a mint-live / redeem-dead market.

The attacker path is concrete and does not require privileged control after the emergency action:

- Operators call in response to a vault malfunction, using the contract's documented emergency path.
- Manager-side USDC approvals to the controller are zeroed and manager vault support is deleted.
- Existing redeem orders for USDC now fail, and any new redeem order for USDC will fail as well.
- But any user who already holds an unexpired signed mint order for USDC can still execute it through so long as the controller's collateral flag has not been independently disabled.
- Supply increases while the redemption leg is already broken, worsening insolvency / redemption failure during the incident window.

This is not a theoretical edge path. Tenbin's mainnet config has a single live collateral entry (), so invoking the emergency removal on the live system would deterministically create this asymmetric state for the entire product unless a second admin action is bundled atomically.

Impact Cascade

- The documented emergency collateral-removal path can immediately brick redemptions for the removed collateral by zeroing manager approvals.
- The same collateral can remain mintable until the controller's separate registry is disabled, allowing unexpired signed mint orders to keep creating fresh.
- Because USDC is the sole live collateral on mainnet, this desync affects the whole system rather than an isolated side market.
- During a vault emergency, the protocol can move into a state where supply expands while users cannot redeem, precisely the opposite of safe containment.

Assumptions and Uncertainties

- The finding assumes operators may rely on as a complete emergency shutdown for a malfunctioning collateral, as its comments imply.
- The exploit window depends on there being outstanding unexpired signed mint orders, which is realistic because order execution is user-triggered and the controller mint path is public.
- Operators can mitigate manually by also pausing the controller or disabling the controller-side collateral flag, but that extra step is not enforced by the contracts.

- The issue is latent until the emergency path is used; once used, the desync is deterministic.

How will the bug recipient respond? “This only happens if admins remove collateral without also pausing the controller or calling . That is an operations runbook issue.”

That response understates the bug. is itself presented as the emergency malfunction response, but it leaves the system in a contradictory state by design. The contracts do not bundle or enforce the follow-up controller action, and the vulnerable mint path remains publicly executable with already-signed orders. A safe emergency primitive should not leave minting live while redemption is broken for the same collateral.

Why did tests miss this issue? The unit tests cover in isolation and controller collateral toggling in isolation, but there is no integration test that removes collateral on the manager and then asserts the controller no longer accepts mints for that collateral. There is likewise no test asserting that emergency removal keeps mint/redeem behavior symmetric across the two contracts.

Recommendation Make collateral disablement atomic across the controller and manager.

Primary fix:

If cross-calling is undesirable, then add an explicit invariant check so and also verify manager support (for example, by consulting or a dedicated support flag) before accepting the collateral. At minimum, make revert unless the controller-side flag has already been disabled in the same administrative workflow.

References

- `src/CollateralManager.sol#L172-L185`
- `src/Controller.sol#L243-L248`
- `src/Controller.sol#L336-L395`
- `src/Controller.sol#L447-L453`
- `src/Controller.sol#L512-L519`
- `README.md#L279-L305`
- `config/mainnet/tgld/tgld.toml#L22-L41`

4.4 Informational Risk

4.4.1 TENB1-3 - README and docs point reviewers to obsolete mainnet Controller and RevenueModule contracts

✓ **Informational**

Status: Open

Original severity: Informational

Executive Summary The repository’s two primary entry points for reviewers (`README.md` and `docs/src/README.md`) advertise the wrong mainnet `Controller` and `RevenueModule` addresses. Both files direct readers to inspect `0xcaF2...` as the controller and `0x5D46...` as the revenue module, while the repository’s own deployment registry (`deployments.json`) records the live mainnet contracts as `0xF24c...` and `0x6DF8...`. I also performed read-only mainnet calls: `AssetToken.minter()` returned `0xF24ceF8227A3126e49Ff272cDC06FC16A97c8967`, and `CollateralManager.revenueModule()` returned `0x6DF8FF2FE0296Fe46b3Cc350EF76E31E388F2256`, confirming the docs are not merely formatted differently - they point to different contracts than the ones actually backing `tgld`.

This is a high-impact hygiene failure because it causes security reviewers, integrators, and operators to trust the wrong production surface. In this repo specifically, the controller is the highest-value trust boundary for mint/redeem authorization, and the revenue module governs the separation between backing collateral and protocol revenue.

Details The top-level `README` and the docs copy both publish the same stale addresses:

```
| Controller | 0xcaF2cD7fd794CaAf56555Db90A5865a5FE9182f7 |
| RevenueModule | 0x5D46Ec01376d218Ade3c1133a7E38976c2DBe584 |
```

But the checked-in deployment registry says the live mainnet contracts are:

```
"controller": "0xF24ceF8227A3126e49Ff272cDC06FC16A97c8967",
"revenue_module": "0x6DF8FF2FE0296Fe46b3Cc350EF76E31E388F2256"
```

The discrepancy is not cosmetic. A read-only mainnet cross-check shows:

- `AssetToken(0x6a54...).minter()` returns `0xF24c...`, so `0xcaF2...` is not the live mint authority.
- `CollateralManager(0x42F3...).revenueModule()` returns `0x6DF8...`, so `0x5D46...` is not the module currently wired into backing-collateral accounting.
- Both the stale and live controller addresses have deployed code, which makes the documentation especially dangerous: a reviewer can easily inspect a real-but-obsolete contract and believe they covered production.

The root cause is that the repo keeps two competing sources of truth for deployments, and the human-facing docs were never updated when the live controller and revenue module changed.

Impact Cascade

- Reviewers can spend their audit effort on the wrong controller - the protocol's most safety-critical contract.
- Integrators can validate signatures, roles, and pause state against a stale controller instead of the live mint authority.
- Revenue-accounting review can target the wrong module, obscuring how live backing USDC is separated from withdrawable revenue.
- Any incident response, monitoring, or operator playbook built from the README can observe or act on obsolete contracts while the live system behaves differently.

Assumptions and Uncertainties

- External reviewers and operators rely on `README.md` / `docs/src/README.md` as intended entry points.
- The checked-in `deployments.json` and live on-chain reads are the intended source of truth for mainnet.
- The issue is an integrity failure in review/deployment metadata; it does not itself require an on-chain code exploit.

How will the bug recipient respond? "`deployments.json` has the right addresses, so careful reviewers should use that instead."

That response does not hold up because the repository front page and docs explicitly present the stale addresses as "Mainnet Addresses". Those are the first links most reviewers and operators will click, and they contradict both the machine-readable deployment registry and live mainnet wiring.

Why did tests miss this issue? The automated suite exercises contract logic, not the repository's human-facing deployment metadata. Nothing asserts that the README address table matches `deployments.json` or live on-chain configuration.

Recommendation Use a single canonical deployment manifest and generate all published address tables from it. At minimum:

- replace the stale `Controller` and `RevenueModule` rows in both readmes with the `deployments.json` values;
- add the missing `oracle_adapter` address to the published mainnet table so reviewers see the full live trust boundary; and
- add a CI check that parses the README/docs tables and fails if they diverge from `deployments.json`.

References

- [README.md#L7-L8](#)
- [README.md#L15](#)
- [README.md#L21](#)
- [docs/src/README.md#L7-L8](#)
- [docs/src/README.md#L15](#)
- [docs/src/README.md#L21](#)
- [deployments.json#L27-L28](#)
- [deployments.json#L31](#)
- [deployments.json#L35](#)